

INSTITUTE OF ENGINEERING

ADVANCED COLLEGE OF ENGINEERING AND MANAGEMENT
KALANKI, KATHMANDU
(AFFILIATED TO TRIBHUVAN UNIVERSITY)



ADVANCED COLLEGE
OF ENGINEERING & MANAGEMENT



ADVANCED COLLEGE
OF ENGINEERING & MANAGEMENT
Affiliated to Tribhuvan University (T.U.)

LAB TITLE: Implementation of
Backtracking Algorithm

SUBJECT: Artificial Intelligence

LAB NO: 02

SUBMITTED BY:

NAME: Ishan Shrestha
ROLL NO: ACE080BCT030
DATE: June 9, 2026

SUBMITTED TO:

Department of Computer
and Electronics
Engineering

Implementation of Backtracking Algorithm

Objective:

1. To understand the concept of backtracking and depth first approach of backtracking.
2. To implement backtracking techniques for finding feasible solutions to constraint-based problems.
3. To study adversarial search strategies and their application.

Theory:

1. **Backtracking Algorithm:** Backtracking is a general technique for solving problems by trying out possible solutions step by step, and undoing (backtracking) whenever you hit a dead end. It's a depth-first search through a decision tree, pruning branches early when a constraint is violated.

Back tracking is essential to solve constraint satisfaction problems, as it's an core to implement solutions to such issues.

The generic schema of it is:

```
function BACKTRACK(assignment, csp):
    if assignment is complete: return assignment
    var ← SELECT-UNASSIGNED-VARIABLE(csp, assignment)
    for each value in ORDER-DOMAIN-VALUES(var, assignment, csp):
        if value is consistent with assignment:
            add {var = value} to assignment
            result ← BACKTRACK(assignment, csp)
            if result ≠ failure: return result
        remove {var = value} from assignment
    return failure
```

2. **Adversarial Search Algorithm:** In adversarial search, the AI agent evaluates all possible moves and counter-moves by the opponent to identify the best strategy. This method ensures that the AI can anticipate the opponent's actions and adjust its approach accordingly.

This applies to two-player competitive games (chess, tic-tac-toe, checkers). The core algorithm is Minimax, one player tries to maximize their score, the other tries to minimize it. Each player assumes the opponent plays optimally.

Alpha-Beta Pruning is an optimization: it skips branches that can't possibly affect the final decision, making Minimax far more efficient.

```
text MINIMAX(s, player):
    if TERMINAL(s): return UTILITY(s)
    if player = MAX:
        return max over a in ACTIONS(s) of
        MINIMAX(RESULT(s,a), MIN)
    if player = MIN:
        return min over a in ACTIONS(s) of MINIMAX(RESULT(s,a), MAX)
```

3. **Constraint Satisfaction Problems:**

A CSP is a triple (X, D, C) :

$X = \{X_1, X_2, \dots, X_n\}$ — variables

$D = \{D_1, D_2, \dots, D_n\}$ — domains, D_i is the set of values for X_i

$C = \{C_1, C_2, \dots, C_m\}$ — constraints, each $C_j = \langle \text{scope}, \text{relation} \rangle$

A solution is a complete assignment satisfying all constraints. A partial assignment is consistent if it violates no constraint.

Source Code:

Backtracking Algorithm

```
import time
import tkinter as tk
from tkinter import messagebox

class TreeNode:
    def __init__(self, name, x, y):
        self.name = name          # Unique Identifier (e.g., 'A', 'B', 'C')
        self.x = x                # Canvas X coordinate
        self.y = y                # Canvas Y coordinate
        self.children = []        # Sub-nodes linked below

class TreeBacktrackDashboard:
    def __init__(self, root):
        self.root = root
        self.root.title("Tree Search & Backtracking Dashboard")
        self.root.geometry("950x650")
        self.root.configure(bg='#1A1A24')

        # Control States
        self.search_target = ""
        self.path_taken = []      # Active tracking stack framework
        self.visited_nodes = set()

        self.create_tree_structure()
        self.create_widgets()

    def create_tree_structure(self):
        """Constructs a deterministic tree with 15 nodes and a depth of 5."""
        # Layer 1: Root (Depth 1)
        self.root_node = TreeNode("Root", 475, 50)

        # Layer 2: (Depth 2)
```

```

b1 = TreeNode("B1", 250, 150)
b2 = TreeNode("B2", 700, 150)
self.root_node.children = [b1, b2]

# Layer 3: (Depth 3)
c1 = TreeNode("C1", 130, 260)
c2 = TreeNode("C2", 370, 260)
c3 = TreeNode("C3", 580, 260)
c4 = TreeNode("C4", 820, 260)
b1.children = [c1, c2]
b2.children = [c3, c4]

# Layer 4: (Depth 4)
d1 = TreeNode("D1", 70, 380)
d2 = TreeNode("D2", 190, 380)
d3 = TreeNode("D3", 310, 380)
d4 = TreeNode("D4", 430, 380)
d5 = TreeNode("D5", 640, 380)
c1.children = [d1, d2]
c2.children = [d3, d4]
c3.children = [d5] # Leaving asymmetry

# Layer 5: (Depth 5)
e1 = TreeNode("E1", 70, 500)
e2 = TreeNode("E2", 430, 500)
d1.children = [e1]
d4.children = [e2]

# Total Nodes count = 1(Root) + 2(B) + 4(C) + 5(D) + 2(E) = 14
nodes.

# Let's add one more node to make it exactly 15 nodes total.
e3 = TreeNode("E3", 640, 500)
d5.children = [e3]

def create_widgets(self):
    # --- Upper Control Bar ---
    control_frame = tk.Frame(self.root, bg='#252538', pady=10)
    control_frame.pack(side=tk.TOP, fill=tk.X)

    tk.Label(
        control_frame, text="Enter Target Node Name (e.g., E2, C3,
D1):",
        font=('Arial', 11, 'bold'), bg='#252538', fg='white'
    ).pack(side=tk.LEFT, padx=(20, 5))

    self.entry_target = tk.Entry(control_frame, font=('Arial',
12), width=8, bg='#121214', fg='white', insertbackground='white')
    self.entry_target.pack(side=tk.LEFT, padx=5)
    self.entry_target.insert(0, "E2") # Default search goal text

```

```

        search_btn = tk.Button(
            control_frame, text="🔍 Search Node (Backtrack)",
            font=('Arial', 11, 'bold'),
            bg='#00E676', fg='black',
            command=self.start_backtrack_search, relief=tk.FLAT
        )
        search_btn.pack(side=tk.LEFT, padx=15)

        reset_btn = tk.Button(
            control_frame, text="🔄 Reset Dashboard View",
            font=('Arial', 11),
            bg='#37474F', fg='white', command=self.reset_display,
            relief=tk.FLAT
        )
        reset_btn.pack(side=tk.LEFT)

        # --- Tree Rendering Canvas ---
        self.canvas = tk.Canvas(self.root, bg='#121214',
            highlightthickness=0)
        self.canvas.pack(fill=tk.BOTH, expand=True, pady=10)

        self.draw_tree(self.root_node)

    def draw_tree(self, node):
        """Recursively renders connections and circle nodes to the
graphics viewport."""
        for child in node.children:
            # Draw structural links
            color = '#37474F'
            # Highlight edge if both parent and child are on the
            # active search path
            if node in self.path_taken and child in self.path_taken:
                color = '#00E676' # Neon green connection path
            elif node in self.visited_nodes and child in
self.visited_nodes:
                color = '#D32F2F' # Dull crimson path for rejected
                configurations

            self.canvas.create_line(node.x, node.y, child.x, child.y,
width=3, fill=color)
            self.draw_tree(child)

        # Render current node node body
        fill_color = '#263238'
        outline_color = '#78909C'

        if node in self.path_taken:
            fill_color = '#00E676' # Green if actively on the
            solution path
            outline_color = '#FFFFFF'

```

```

        elif node.name in self.visited_nodes:
            fill_color = '#D32F2F' # Red if it was a dead-end and we
backtracked off it

            self.canvas.create_oval(node.x-22, node.y-22, node.x+22,
node.y+22, fill=fill_color, outline=outline_color, width=2)
            text_color = 'white' if fill_color != '#00E676' else 'black'
            self.canvas.create_text(node.x, node.y, text=node.name,
font=('Arial', 10, 'bold'), fill=text_color)

# --- Core Backtracking Algorithm ---
def backtrack_search(self, current_node):
    # 1. Update UI state to show node entry
    self.path_taken.append(current_node)
    self.visited_nodes.add(current_node.name)

    self.refresh_canvas_view(250) # short execution pause to
animate search

    # Base Case / Goal Test Success Check
    if current_node.name.upper() == self.search_target:
        return True

    # 2. Iterate through choices (Child branches)
    for child in current_node.children:
        # Recurse down the branch
        if self.backtrack_search(child):
            return True # Success propagation upward

    # 3. BACKTRACK STEP: If children did not find the target, this
path is a dead end.
    # We pop the node off our active solution stack to step back
up to the parent.
    self.path_taken.pop()
    self.refresh_canvas_view(250) # short execution pause to
animate backtracking

    return False

def start_backtrack_search(self):
    self.search_target = self.entry_target.get().strip().upper()
    self.path_taken = []
    self.visited_nodes.clear()

    if not self.search_target:
        messagebox.showwarning("Input Error", "Please provide a
valid node key target.")
        return

    found = self.backtrack_search(self.root_node)

```

```

        if found:
            path_str = " -> ".join([node.name for node in
self.path_taken])
            messagebox.showinfo("Target Found!", f"Successfully
located node {self.search_target}!\n\nBacktracked Verification Path:\
n{path_str}")
        else:
            messagebox.showerror("Not Found", f"Node
'{self.search_target}' does not exist inside this tree topology
structure.")

    def refresh_canvas_view(self, ms_delay):
        self.canvas.delete("all")
        self.draw_tree(self.root_node)
        self.root.update()
        self.root.after(ms_delay)

    def reset_display(self):
        self.path_taken.clear()
        self.visited_nodes.clear()
        self.canvas.delete("all")
        self.draw_tree(self.root_node)

if __name__ == "__main__":
    main_window = tk.Tk()
    app = TreeBacktrackDashboard(main_window)
    main_window.mainloop()

```

Adversial Search Algorithm

```

import math
import tkinter as tk
from tkinter import messagebox

class TicTacToeGUI:
    def __init__(self, root):
        self.root = root
        self.root.title("AI Tic-Tac-Toe (Minimax)")

        self.ai = 'O'
        self.human = 'X'
        self.board = [' ' for _ in range(9)]

        self.buttons = []
        self.create_widgets()

    def create_widgets(self):

```

```

# Game grid configuration
for i in range(9):
    # The lambda captures the current value of i as idx
    btn = tk.Button(
        self.root, text=' ', font=('Arial', 24, 'bold'),
        width=5, height=2, bg='#f0f0f0',
        command=lambda idx=i: self.human_move(idx)
    )
    # FIXED: Changed 'idx' to 'i' to match the loop iterator
    row = i // 3
    col = i % 3
    btn.grid(row=row, column=col, padx=5, pady=5)
    self.buttons.append(btn)

# Reset Button
reset_btn = tk.Button(
    self.root, text="Reset Game", font=('Arial', 12),
    command=self.reset_game, bg='#d9e2ec'
)
reset_btn.grid(row=3, column=0, columnspan=3, sticky="s",
pady=10)

def human_move(self, index):
    if self.board[index] == ' ' and not self.check_winner(self.ai)
and not self.check_winner(self.human):
        self.board[index] = self.human
        self.buttons[index].config(text=self.human,
state='disabled', disabledforeground='#1976D2')

        if self.evaluate_game_over():
            return

        # Trigger AI Turn immediately after a tiny delay
        self.root.after(300, self.ai_move)

def ai_move(self):
    best_move = self.find_best_move()
    if best_move is not None:
        self.board[best_move] = self.ai
        self.buttons[best_move].config(text=self.ai,
state='disabled', disabledforeground='#D32F2F')
        self.evaluate_game_over()

def find_best_move(self):
    best_score = -math.inf
    best_move = None
    for move in self.get_available_moves():
        self.board[move] = self.ai
        score = self.minimax(-math.inf, math.inf, False)

```



```

        self.board[move] = ' '
        if score > best_score:
            best_score = score
            best_move = move
    return best_move

def minimax(self, alpha, beta, is_maximizing):
    if self.check_winner(self.ai): return 10
    if self.check_winner(self.human): return -10
    if ' ' not in self.board: return 0

    if is_maximizing:
        max_eval = -math.inf
        for move in self.get_available_moves():
            self.board[move] = self.ai
            evaluation = self.minimax(alpha, beta, False)
            self.board[move] = ' '
            max_eval = max(max_eval, evaluation)
            alpha = max(alpha, evaluation)
            if beta <= alpha: break
        return max_eval
    else:
        min_eval = math.inf
        for move in self.get_available_moves():
            self.board[move] = self.human
            evaluation = self.minimax(alpha, beta, True)
            self.board[move] = ' '
            min_eval = min(min_eval, evaluation)
            beta = min(beta, evaluation)
            if beta <= alpha: break
        return min_eval

def get_available_moves(self):
    return [i for i, spot in enumerate(self.board) if spot == ' ']

def check_winner(self, player):
    win_conditions = [
        [0,1,2], [3,4,5], [6,7,8], [0,3,6],
        [1,4,7], [2,5,8], [0,4,8], [2,4,6]
    ]
    return any(all(self.board[pos] == player for pos in cond) for
cond in win_conditions)

def evaluate_game_over(self):
    if self.check_winner(self.human):
        messagebox.showinfo("Game Over", "Unbelievable! You won.")
        return True
    elif self.check_winner(self.ai):
        messagebox.showinfo("Game Over", "AI wins! Minimax is
flawless.")

```

```

        return True
    elif ' ' not in self.board:
        messagebox.showinfo("Game Over", "It's a draw!")
        return True
    return False

def reset_game(self):
    self.board = [' ' for _ in range(9)]
    for btn in self.buttons:
        btn.config(text=' ', state='normal', bg='#f0f0f0')

if __name__ == "__main__":
    main_window = tk.Tk()
    app = TicTacToeGUI(main_window)
    main_window.mainloop()

```

Constraint Satisfaction Algorithm

```

import math
import random
import tkinter as tk
from tkinter import messagebox

class DynamicCSP10NodesGUI:
    def __init__(self, root):
        self.root = root
        self.root.title("Dynamic CSP Playground (10 Nodes Layout)")
        self.root.geometry("900x650")
        self.root.configure(bg='#1e1e24')

        # Colors available for selection (Palette)
        self.colors = ['#FF5252', '#4CAF50', '#2196F3', '#FFEB3B'] #
        Red, Green, Blue, Yellow
        self.color_names = ['Red', 'Green', 'Blue', 'Yellow']
        self.selected_color = self.colors[0]

        # CSP Variables
        self.num_nodes = 10
        self.nodes = [f"N{i}" for i in range(1, self.num_nodes + 1)]
        self.coords = {} # {'N1': (x, y)}
        self.topology = {} # {'N1': ['N2', 'N3']}
        self.assignments = {} # Active solutions: {'N1': '#FF5252'}

        self.create_widgets()
        self.generate_10_node_graph()

    def create_widgets(self):
        # --- Top Control Panel ---

```

```

control_frame = tk.Frame(self.root, bg='#2a2a35', pady=10)
control_frame.pack(side=tk.TOP, fill=tk.X)

gen_btn = tk.Button(
    control_frame, text="🗺️ Generate New 10-Node Map",
    font=('Arial', 11, 'bold'),
    bg='#9C27B0', fg='white',
    command=self.generate_10_node_graph, relief=tk.FLAT
)
gen_btn.pack(side=tk.LEFT, padx=15)

solve_btn = tk.Button(
    control_frame, text="🧠 AI Auto-Solve", font=('Arial', 11,
'bold'),
    bg='#4CAF50', fg='white', command=self.run_ai_solver,
    relief=tk.FLAT
)
solve_btn.pack(side=tk.LEFT, padx=5)

clear_btn = tk.Button(
    control_frame, text="🧼 Clear Colors", font=('Arial', 11),
    bg='#555566', fg='white', command=self.clear_assignments,
    relief=tk.FLAT
)
clear_btn.pack(side=tk.LEFT, padx=5)

# --- Side Color Selection Palette ---
palette_frame = tk.Frame(self.root, bg='#2a2a35', width=150,
    padx=15, pady=20)
palette_frame.pack(side=tk.LEFT, fill=tk.Y)

palette_label = tk.Label(
    palette_frame, text="SELECT COLOR", font=('Arial', 10,
'bold'),
    bg='#2a2a35', fg='#B0BEC5'
)
palette_label.pack(anchor=tk.W, pady=(0, 15))

self.color_var = tk.StringVar(value=self.colors[0])
for i, color in enumerate(self.colors):
    frame = tk.Frame(palette_frame, bg='#2a2a35', pady=6)
    frame.pack(fill=tk.X)

    rb = tk.Radiobutton(
        frame, text=self.color_names[i],
        variable=self.color_var,
        value=color, bg='#2a2a35', fg='white',
        selectcolor='#2a2a35',
        font=('Arial', 11), command=self.update_selected_color
    )

```

```

        rb.pack(side=tk.LEFT)

        canvas_indicator = tk.Canvas(frame, width=20, height=20,
bg=color, highlightthickness=0)
        canvas_indicator.pack(side=tk.RIGHT, padx=5)

        # --- Graphics View Canvas ---
        self.canvas = tk.Canvas(self.root, bg='#121214',
highlightthickness=0)
        self.canvas.pack(side=tk.RIGHT, fill=tk.BOTH, expand=True)
        self.canvas.bind("<Button-1>", self.handle_canvas_click)

    def generate_10_node_graph(self):
        """Generates a stable, non-overlapping 10-node layout using
Grid Anchors."""
        self.assignments.clear()
        self.coords.clear()
        self.topology.clear()

        # Define 12 possible logical sectors on canvas (4 columns x 3
rows grid map)
        cols, rows = 4, 3
        cell_width = 700 // cols
        cell_height = 550 // rows

        sectors = [(c, r) for c in range(cols) for r in range(rows)]
        # Pick 10 random sectors out of the 12 to place our 10 nodes
        chosen_sectors = random.sample(sectors, self.num_nodes)

        # Generate coordinates with local jitter inside their sector
bounds
        for idx, node in enumerate(self.nodes):
            sc_col, sc_row = chosen_sectors[idx]

            # Compute sector center coordinates
            base_x = int((sc_col + 0.5) * cell_width) + 30
            base_y = int((sc_row + 0.5) * cell_height) + 30

            # Inject a small random offset (jitter) to keep the layout
feeling dynamic
            x = base_x + random.randint(-20, 20)
            y = base_y + random.randint(-20, 20)

            self.coords[node] = (x, y)
            self.topology[node] = []

        # Generate a realistic tree/mesh topology network
        for i, node in enumerate(self.nodes):
            # Sort all other nodes by physical distance from current
node

```

```

        distances = [(target, math.hypot(self.coords[node][0] -
self.coords[target][0],
                                         self.coords[node][1] -
self.coords[target][1]))
        for target in self.nodes if target != node]
        distances.sort(key=lambda item: item[1])

        # Force connect to closest neighbor to guarantee no node
is isolated
        closest_neighbor = distances[0][0]
        if closest_neighbor not in self.topology[node]:
            self.topology[node].append(closest_neighbor)
            self.topology[closest_neighbor].append(node)

        # Randomly attach to secondary neighbors to build complex
constraints
        num_extra_edges = random.randint(1, 2)
        for j in range(1, min(num_extra_edges + 1,
len(distances))):
            candidate = distances[j][0]
            # Enforce max degree limits (max 3 edges per node) to
maintain planar colorability
            if len(self.topology[node]) < 3 and
len(self.topology[candidate]) < 3:
                if candidate not in self.topology[node]:
                    self.topology[node].append(candidate)
                    self.topology[candidate].append(node)

        self.draw_graph()

    def draw_graph(self):
        self.canvas.delete("all")

        # Draw constraints edges
        drawn_edges = set()
        for node, neighbors in self.topology.items():
            for neighbor in neighbors:
                edge = tuple(sorted((node, neighbor)))
                if edge not in drawn_edges:
                    x1, y1 = self.coords[node]
                    x2, y2 = self.coords[neighbor]
                    self.canvas.create_line(x1, y1, x2, y2, width=3,
fill='#37474F')
                    drawn_edges.add(edge)

        # Draw structural node bodies
        for node, (x, y) in self.coords.items():
            color = self.assignments.get(node, '#3A3A4A') # Dark gray
if unassigned

```

```

        # Nodes rendering radius set to 22 pixels
        self.canvas.create_oval(x-22, y-22, x+22, y+22,
fill=color, outline='#78909C', width=2)
        self.canvas.create_text(x, y, text=node, font=('Arial',
10, 'bold'), fill='white')

    def update_selected_color(self):
        self.selected_color = self.color_var.get()

    def handle_canvas_click(self, event):
        """Processes human node interaction and triggers adjacency
logic confirmation checks."""
        clicked_node = None
        for node, (x, y) in self.coords.items():
            if math.hypot(event.x - x, event.y - y) <= 22:
                clicked_node = node
                break

        if clicked_node:
            if self.is_consistent(clicked_node, self.selected_color):
                self.assignments[clicked_node] = self.selected_color
                self.draw_graph()
            else:
                messagebox.showwarning(
                    "Constraint Violation",
                    f"Conflict detected! An adjacent linked node is
already colored {self.get_color_name(self.selected_color)}."
                )

    def get_color_name(self, hex_code):
        return self.color_names[self.colors.index(hex_code)]

    # --- Backtracking CSP Engine Loop Core ---
    def is_consistent(self, variable, value):
        for neighbor in self.topology.get(variable, []):
            if neighbor in self.assignments and
self.assignments[neighbor] == value:
                return False
        return True

    def select_mrv_variable(self):
        unassigned = [v for v in self.nodes if v not in
self.assignments]
        if not unassigned: return None

        def count_legal_values(var):
            return sum(1 for val in self.colors if
self.is_consistent(var, val))

        return min(unassigned, key=count_legal_values)

```

```

def backtrack(self):
    if len(self.assignments) == len(self.nodes):
        return True

    var = self.select_mrv_variable()
    if var is None: return False

    for value in self.colors:
        if self.is_consistent(var, value):
            self.assignments[var] = value

            # Dynamic refresh pipeline interface
            self.root.update()
            self.root.after(80) # slightly faster calculation
            playback for 10 nodes scale
            self.draw_graph()

            if self.backtrack():
                return True

            del self.assignments[var]
            self.draw_graph()

    return False

def run_ai_solver(self):
    self.assignments.clear()
    if self.backtrack():
        messagebox.showinfo("Solver Complete", "Success! The AI
Backtracking algorithm resolved all 10 nodes cleanly.")
    else:
        messagebox.showerror("Solver Error", "Failed to compute a
valid mapping solution.")

def clear_assignments(self):
    self.assignments.clear()
    self.draw_graph()

if __name__ == "__main__":
    main_window = tk.Tk()
    app = DynamicCSP10NodesGUI(main_window)
    main_window.mainloop()

```

Discussion:

In this lab, we explored three key AI techniques: Backtracking, Adversarial Search, and Constraint Satisfaction Problems (CSPs). These methods are designed to solve complex problems more efficiently than brute-force approaches.

Backtracking uses a depth-first search strategy to explore possible solutions and discard invalid paths when constraints are violated. It is widely used in problems such as N-Queens, Sudoku, and maze solving. While effective, its performance can decrease as problem size grows.

Adversarial Search focuses on decision-making in competitive scenarios. Using the Minimax algorithm, one player aims to maximize outcomes while the opponent seeks to minimize them. Alpha-Beta pruning further improves efficiency by eliminating unnecessary branches of the search tree.

CSPs represent problems through variables, domains, and constraints. Solutions are found by assigning values that satisfy all constraints, often using backtracking. Techniques such as forward checking help reduce the search space and enhance overall performance.

Conclusion:

This lab demonstrated the importance of Backtracking, Adversarial Search, and Constraint Satisfaction Problems (CSPs) as fundamental AI problem-solving techniques. Backtracking enables systematic exploration of solution spaces, adversarial search supports decision-making in competitive environments, and CSPs offer an effective framework for solving constraint-based problems.

The lab also highlighted the role of optimization methods, such as pruning and constraint propagation, in improving efficiency and reducing computational complexity. Overall, it provided valuable insight into AI search algorithms and their practical applications across various domains.